

Manual de Usuario – EffiCode Analyzer

Guía de Uso del Sistema de Análisis de Complejidad Algorítmica

Versión: 1.0.0

Fecha: Diciembre 2025

Curso: Análisis y Diseño de Algoritmos

Universidad: Universidad del Caldas

Tabla de Contenidos

1. [Introducción](#)
2. [Requisitos Previos](#)
3. [Iniciar la Aplicación](#)
4. [Interfaz de Usuario](#)
5. [Funcionalidades](#)
6. [Ejemplos de Uso](#)
7. [Preguntas Frecuentes](#)

1. Introducción

EffiCode Analyzer es una herramienta web que permite analizar la complejidad algorítmica de pseudocódigo escrito en formato Cormen (del libro *"Introduction to Algorithms"*).

¿Qué puede hacer?

Función	Descripción
Calcular complejidad	Determina O (peor caso), Ω (mejor caso) y Θ (caso promedio)
Mostrar pasos	Explica paso a paso cómo se resolvió el análisis (sumatorias, recurrencias)
Visualizar AST	Genera el árbol sintáctico abstracto del algoritmo
Validación IA	Google Gemini verifica el resultado usando el libro de Cormen
Generar PDF	Exporta un reporte completo del análisis
Lenguaje Natural	Traduce descripciones en español a pseudocódigo

2. Requisitos Previos

Antes de usar la aplicación, asegúrese de que:

- El **Backend** esté ejecutándose en `http://localhost:8000`
- El **Frontend** esté ejecutándose en `http://localhost:5173`
- Tenga conexión a **Internet** (es necesaria para la validación con IA)

Consulte el **Manual Técnico** para instrucciones detalladas de instalación y ejecución.

3. Iniciar la Aplicación

Paso 1: Iniciar el Backend

Abra una terminal en el directorio `/Backend` y ejecute:

```
source venv/bin/activate  
python main_api.py
```

Debería ver:

```
Services initialized successfully.  
INFO: Uvicorn running on http://0.0.0.0:8000
```

Paso 2: Iniciar el Frontend

En otra terminal, en el directorio `/Frontend`:

```
npm run dev
```

Debería ver:

```
VITE v7.2.4 ready  
→ Local: http://localhost:5173/
```

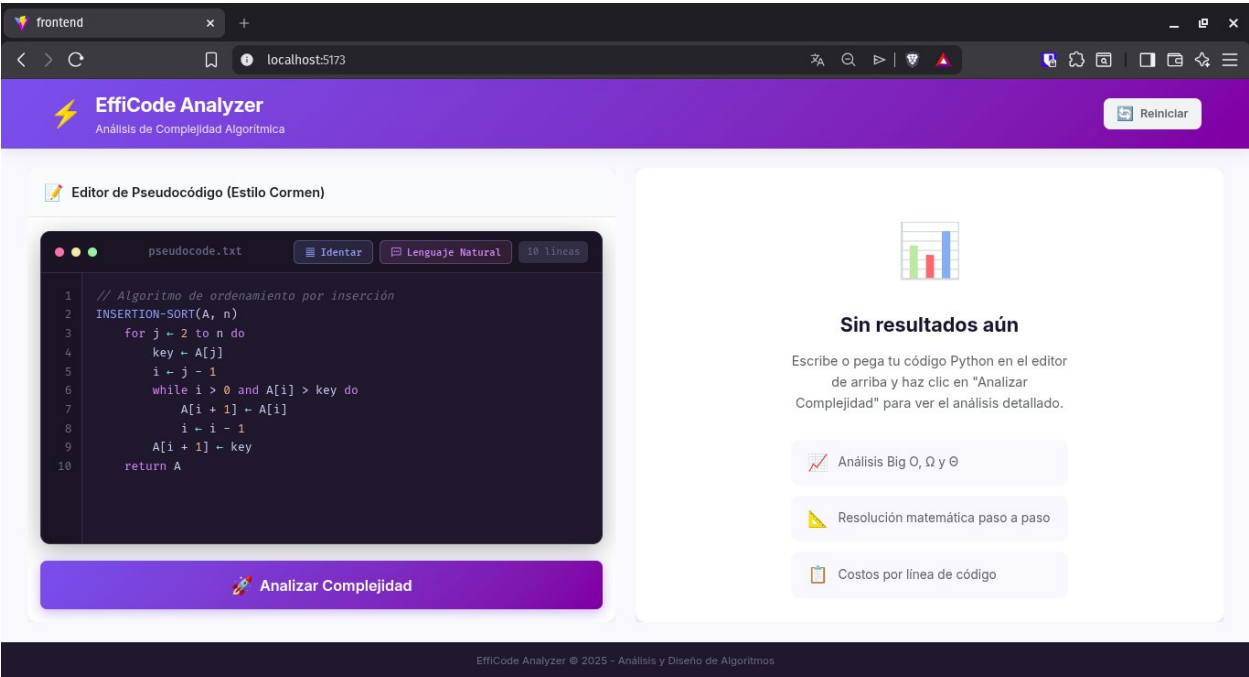
Paso 3: Acceder a la Aplicación

Abra su navegador web y vaya a:

<http://localhost:5173>

```
VITE v7.2.4 ready in 2557 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```



4. Interfaz de Usuario

4.1 Vista General

La interfaz se divide en dos paneles principales (Editor y Resultados):

Componente	Ubicación	Función
Editor de Código	Panel izquierdo	Área para escribir pseudocódigo
Botón Analizar	Debajo del editor	Inicia el análisis de complejidad

Botón Lenguaje Natural	Debajo del editor	Abre modal para traducir español a pseudocódigo
Tarjetas de Complejidad	Panel derecho (arriba)	Muestra O, Omega, Theta
Pestañas de Resultados	Panel derecho	Diferentes vistas del análisis (Pasos, Costos, AST, IA)

pseudocode.txt
Identar
Lenguaje Natural
10 líneas

```

1  // Algoritmo de ordenamiento por inserción
2  INSERTION-SORT(A, n)
3      for j ← 2 to n do
4          key ← A[j]
5          i ← j - 1
6          while i > 0 and A[i] > key do
7              A[i + 1] ← A[i]
8              i ← i - 1
9          A[i + 1] ← key
10     return A

```


Analizar Complejidad

Lenguaje Natural



Complejidad



Peor Caso



Mejor Caso



Caso Promedio



Costos



Validación



Descargar



AST



BIG O (PEOR CASO)

$$O(n^2)$$

Cuadrática

Bucles anidados, crece rápido



BIG Ω (MEJOR CASO)

$$\Omega(n)$$

Lineal

Proporcional al tamaño de entrada



E[T(N)] (CASO PROMEDIO)

$$E[T(n)] =$$

Complejidad

Peor Caso

Mejor Caso

Caso Promedio

Costos

Validación

Descargar

AST

Resolución Matemática - Caso Promedio $E[T(n)]$

1

Paso 1: Definir la Distribución de Entrada (Cormen, Cap. 5.2)

Asumimos que la entrada es una permutación aleatoria uniforme. Cada una de las $n!$ permutaciones tiene probabilidad $1/n!$.

$$Pr\{\pi\} = \frac{1}{n!}, \quad \forall \pi \in S_n \text{ (grupo simétrico)}$$

💡

Sin esta definición, el "caso promedio" no existe matemáticamente. Es el supuesto estándar para algoritmos de ordenamiento.

5. Funcionalidades

5.1 Escribir Pseudocódigo

El editor acepta pseudocódigo en formato **Cormen**. La sintaxis clave incluye:

Elemento	Sintaxis	Ejemplo
Nombre de función	NOMBRE(parámetros)	INSERTION-SORT(A, n)
Asignación	$\leftarrow$	key $\leftarrow$ A[j]
Bucle for	for var $\leftarrow$ inicio to fin do	for j $\leftarrow$ 2 to n do

Bucle while	while condición do	while $i > 0$ do
Condicional	if condición then ... else	if $A[i] > \text{key}$ then

Ejemplo de código válido:

```

INSERTION-SORT(A, n)
  for j ← 2 to n do
    key ← A[j]
    i ← j - 1
    while i > 0 and A[i] > key do
      A[i + 1] ← A[i]
      i ← i - 1
    A[i + 1] ← key
  return A

```

5.2 Analizar Pseudocódigo

1. Escriba o pegue su pseudocódigo en el editor.
2. Haga clic en el botón **Analizar**.
3. Espere a que el indicador de carga desaparezca.



Analizar Complejidad

5.3 Interpretar Resultados

5.3.1 Tarjetas de Complejidad

Tarjeta	Notación	Significado
Peor Caso	O	Cota superior – máximo tiempo posible

Mejor Caso	Omega	Cota inferior – mínimo tiempo posible
Caso Promedio	Theta	Cota ajustada – comportamiento típico

 Complejidad
  Peor Caso
  Mejor Caso
  Caso Promedio
  Costos

 Validación
  Descargar
  AST

 BIG O (PEOR CASO)

$$O(n^2)$$

Cuadrática

Bucles anidados, crece rápido

 BIG Ω (MEJOR CASO)

$$\Omega(n)$$

Lineal

Proporcional al tamaño de entrada

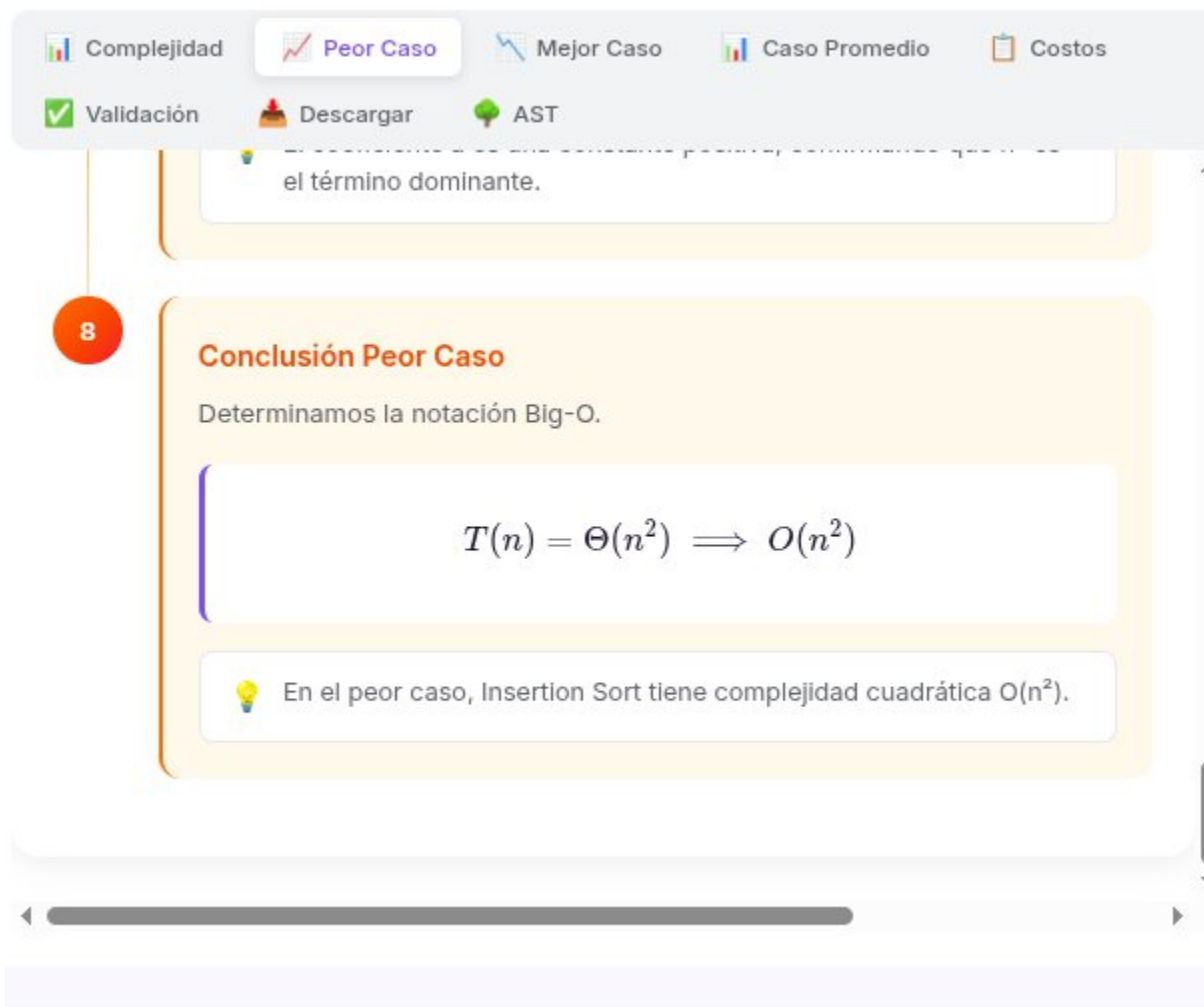
 E[T(N)] (CASO PROMEDIO)

$$E[T(n)] =$$

5.3.2 Pestaña "Peor Caso" – Pasos de Resolución

Muestra la derivación matemática de $T(n)$:

1. **Identificación** de bucles y estructuras.
2. **Construcción** de la función $T(n)$.
3. **Resolución** de sumatorias o recurrencias.
4. **Conclusión** con la notación asintótica.



Complejidad Peor Caso Mejor Caso Caso Promedio Costos

Validación Descargar AST

el término dominante.

8

Conclusión Peor Caso

Determinamos la notación Big-O.

$$T(n) = \Theta(n^2) \implies O(n^2)$$

💡 En el peor caso, Insertion Sort tiene complejidad cuadrática $O(n^2)$.

5.3.3 Pestaña "Mejor Caso"

Muestra el análisis bajo condiciones óptimas (ej: arreglo ya ordenado).

 Complejidad

 Peor Caso

 **Mejor Caso**

 Caso Promedio

 Costos

 Validación

 Descargar

 AST

 El término lineal domina, la constante se vuelve despreciable.

6

Conclusión Mejor Caso

Determinamos la notación Omega.

$$T(n) = \Theta(n) \implies \Omega(n)$$

 En el mejor caso, Insertion Sort tiene complejidad lineal $\Omega(n)$.

5.3.4 Pestaña "Costos por Línea"

Tabla detallada del costo de cada instrucción.

 Complejidad
  Peor Caso
  Mejor Caso
  Caso Promedio

 Costos

 Validación
  Descargar
  AST

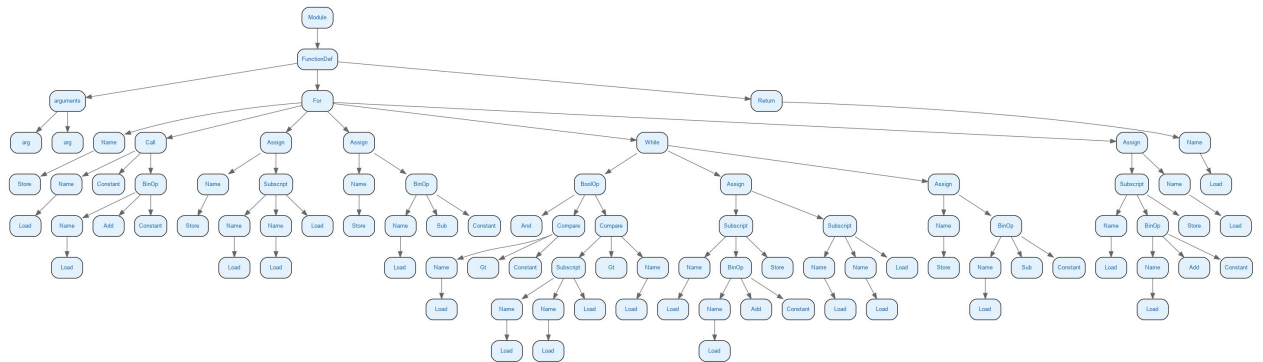


Costo por Línea

Línea	Descripción	Costo
3	Inicialización y test del bucle for (j)	c_1
4	(Dentro de for j) Asignación	$\sum_{j=2}^n c_2$
5	(Dentro de for j) Asignación	$\sum_{j=2}^n c_3$
6	(Dentro de for j) Test de condición while	$\sum_{j=2}^n c_4$
7	(Dentro de for j) (Dentro de for j) Asignación	$\sum_{1 \leq j \leq n} \sum_{2 \leq j \leq n} c_5$
8	(Dentro de for j) (Dentro de for j) Asignación	$\sum_{1 \leq i \leq n} \sum_{2 \leq i \leq n} c_6$

5.3.5 Pestaña "AST" – Árbol Sintáctico

Visualización gráfica del Árbol Sintáctico Abstracto.



5.3.6 Pestaña "Validación IA"

Muestra la \textbf{explicación textual} de Google Gemini.



Validación con IA

✓ VALIDADO

Validación del Análisis.

El análisis propuesto es **correcto y demuestra una sólida comprensión** de la complejidad del algoritmo de ordenamiento por inserción, tal como se presenta en **Introduction to Algorithms**.

Mi evaluación detallada es la siguiente:

1

Peor Caso ($O(n^2)$)

Correcto. Como se describe en el texto (Capítulo 2), el peor caso ocurre cuando el array está en orden inverso. Para cada elemento j , el bucle interno `while` debe compararlo y desplazar todos los $j-1$ elementos

5.3.7 Pestaña "Descargar" – Reporte PDF

Descarga un informe completo del análisis.

Caso	Notación	Descripción
Peor Caso (O)	$O(n^2)$	Cota superior asintótica
Mejor Caso (Ω)	$\Omega(n)$	Cota inferior asintótica
Caso Ajustado (Θ)	No aplicable	Cota ajustada
Caso Promedio $E[T(n)]$	$E[T(n)] = \Theta(n^2)$	Tiempo esperado

3. Análisis del Peor Caso

Paso 1: Función de tiempo $T(n)$ - Peor Caso

Sumamos los costos de cada línea del algoritmo. En el peor caso (array en orden inverso), el bucle while se ejecuta $j-1$ veces para cada j .

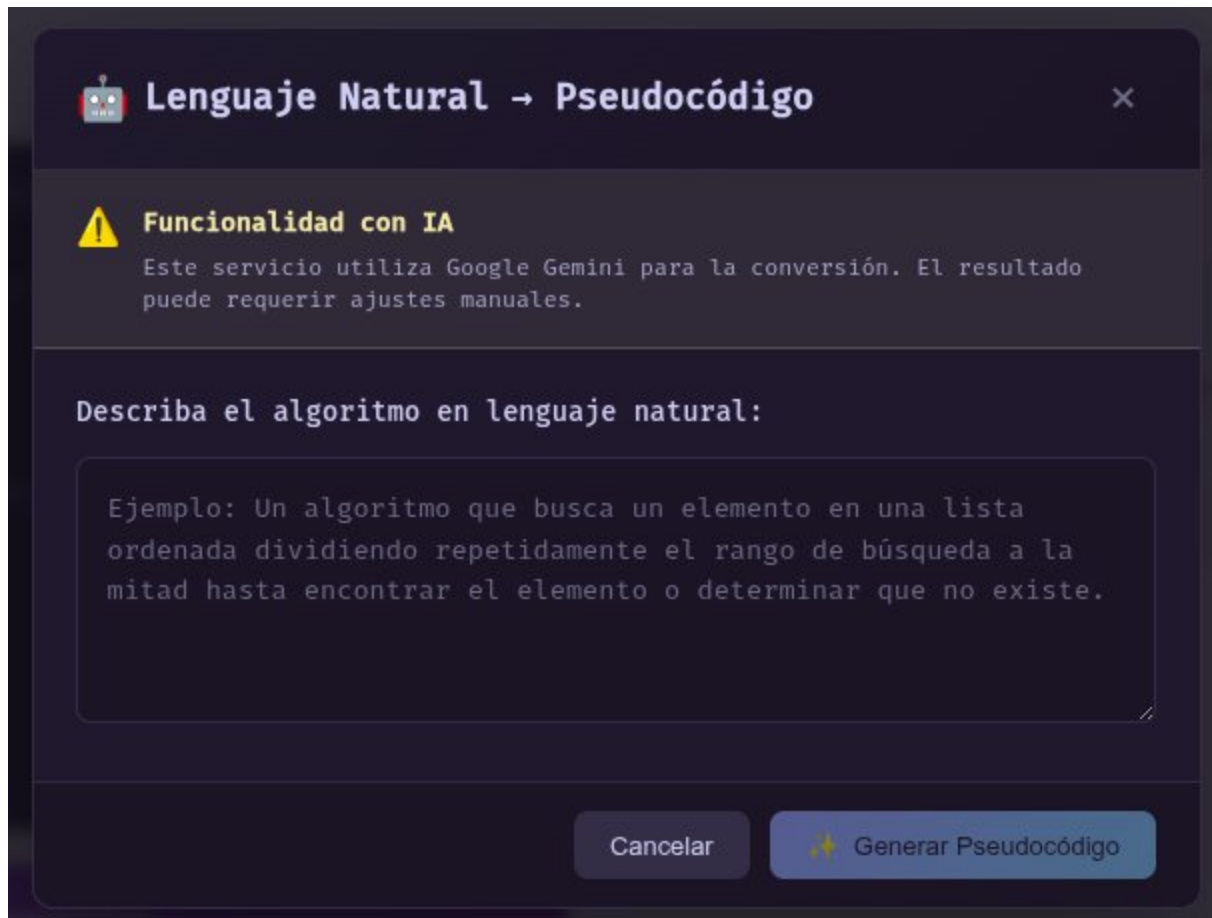
$$T(n) = c_1 \cdot n + c_2(n-1) + c_4(n-1) + c_3 \sum_{j=2}^n j + c_5 \sum_{j=2}^n (j-1) + c_6 \sum_{j=2}^n (j-1) + c_7(n-1)$$

Cada línea contribuye: bucle for ($c_1 \cdot n$), asignaciones (c_2, c_4, c_7 ejecutadas $n-1$ veces), y el while con sus operaciones internas que dependen de j .

5.4 Traducción de Lenguaje Natural

Pasos:

1. Clic en **Lenguaje Natural**.
2. Escriba la descripción en español (ej: "ordenar una lista usando el método de la burbuja").
3. Clic en **Traducir**.
4. El pseudocódigo generado se insertará en el editor.



6. Ejemplos de Uso

6.1 Ejemplo 1: Insertion Sort (Algoritmo Iterativo)

Código:

```
// Algoritmo de ordenamiento por inserción
INSERTION-SORT(A, n)
  for j ← 2 to n do
    key ← A[j]
    i ← j - 1
    while i > 0 and A[i] > key do
      A[i + 1] ← A[i]
      i ← i - 1
    A[i + 1] ← key
  return A
```

Resultados esperados: $O(n^2)$, $\Omega(n)$, $\Theta(n^2)$.

Editor de Pseudocódigo (Estilo Corman)

```
1 // Algoritmo de ordenamiento por inserción
2 INSERTION-SORT(A, n)
3   for j ← 2 to n do
4     key ← A[j]
5     i ← j - 1
6     while i > 0 and A[i] > key do
7       A[i + 1] ← A[i]
8       i ← i - 1
9     A[i + 1] ← key
10  return A
```

Analizar Complejidad

Complejidad Peor Caso Mejor Caso Caso Promedio Costos

Validación Descargar AST

BIG O (PEOR CASO)

$O(n^2)$

Cuadrática

Bucles anidados, crece rápido

BIG Ω (MEJOR CASO)

$\Omega(n)$

Lineal

Proporcional al tamaño de entrada

E[T(N)] (CASO PROMEDIO)

$E[T(n)] =$

Complejidad Peor Caso Mejor Caso **Caso Promedio** Costos

Validación Descargar AST

8

Resumen Metodológico

Lista de chequeo completada según Corman Cap. 5:

$E[T(n)] = \Theta(n^2)$

✓ Distribución definida (1/n!) ✓ Variable aislada ✓ Indicadoras ✓ Probabilidad local ✓ Linealidad aplicada

6.2 Ejemplo 2: Merge Sort (Algoritmo Recursivo)

Código:

```
MERGE-SORT(A, p, r)

    if  $p \geq r$            // cero o un elemento

        return

     $q = \lfloor (p + r) / 2 \rfloor$  // punto medio de A[p : r]

    MERGE-SORT(A, p, q) // ordenar A[p : q]

    MERGE-SORT(A, q + 1, r) // ordenar A[q + 1 : r]

    MERGE(A, p, q, r) // mezclar ambas mitades
```

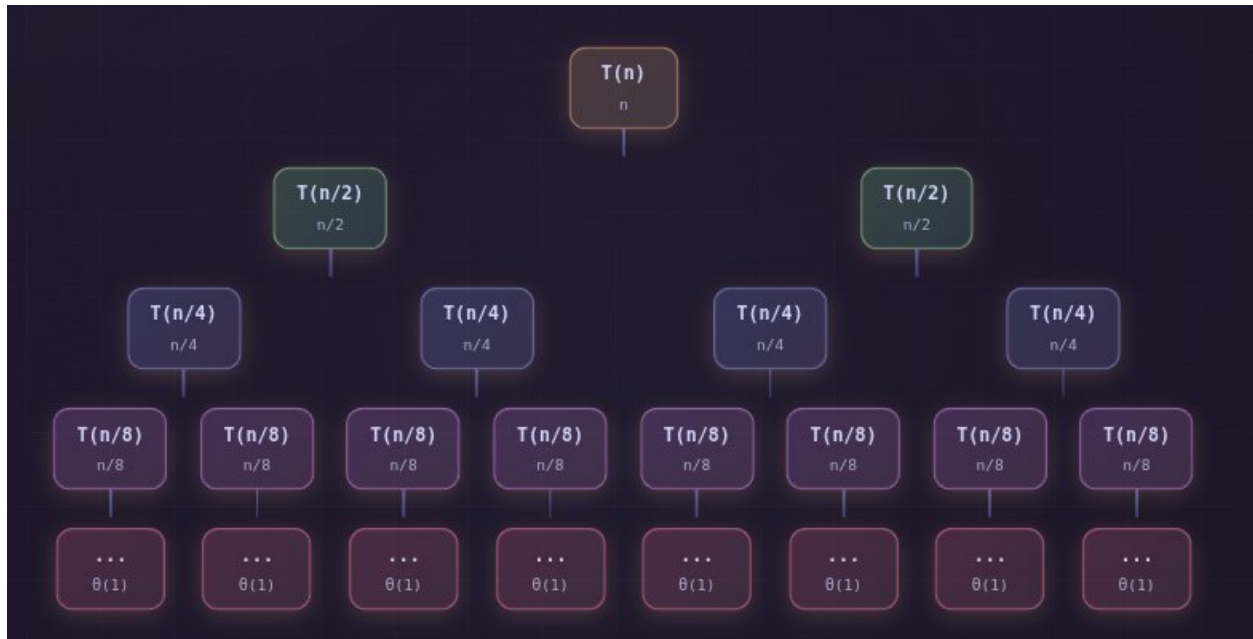
Resolución: Resuelve $T(n) = 2T(n/2) + n$ usando Teorema Maestro (Caso 2).

The image shows a screenshot of a web-based tool for analyzing the complexity of an algorithm. On the left is a 'Pseudocode Editor (Cormen Style)' with a dark theme. It contains the Merge Sort algorithm code, with line numbers 1 through 8. The code is as follows:

```
1 MERGE-SORT(A, p, r)
2   if  $p \geq r$            // cero o un elemento
3       return
4    $q = \lfloor (p + r) / 2 \rfloor$  // punto medio de A[p : r]
5   MERGE-SORT(A, p, q) // ordenar A[p : q]
6   MERGE-SORT(A, q + 1, r) // ordenar A[q + 1 : r]
7   MERGE(A, p, q, r) // mezclar ambas mitades
8
```

Below the editor is a purple button labeled 'Analizar Complejidad'. On the right is a panel titled 'Complejidad' (Complexity) with several tabs: 'Complejidad', 'Peor Caso', 'Mejor Caso', and 'Caso Promedio'. The 'Complejidad' tab is active, showing a grid of complexity results:

- BIG O (PEOR CASO):** $O(n \lg n)$, Logarítmica, Crece muy lentamente.
- BIG Ω (MEJOR CASO):** $\Omega(n \lg n)$, Logarítmica, Crece muy lentamente.
- BIG Θ (COTA AJUSTADA):** $\Theta(n \lg n)$.
- E[T(N)] (CASO PROMEDIO):** $E[T(n)] =$



Complejidad

Peor Caso

Mejor Caso

Caso Promedio

Árbol Recursión

Costos

Validación

Descargar

AST

1

Identificar forma de la recurrencia

****Teorema Maestro**** para $T(n) = 2T(n/2) + n$

$$T(n) = 2T(n/2) + n$$

Aplicando el Teorema Maestro de Cormen Cap. 4.5

2

Aplicar Teorema Maestro

Aplicando el Teorema Maestro de Cormen Cap. 4.5

7. Preguntas Frecuentes

? ¿Por qué el análisis tarda mucho?

La primera vez que se ejecuta, el sistema carga el libro de Cormen en la IA, lo cual puede tardar 10–30 segundos. Las siguientes consultas serán más rápidas gracias al caché.

? ¿El sistema funciona sin Internet?

El análisis básico (parsing, AST, y cálculo simbólico Theta) funciona sin internet. Sin embargo, la **validación con IA** y la **traducción de Lenguaje Natural** requieren conexión.

? ¿Qué hago si aparece un error de sintaxis?

Verifique que su pseudocódigo sigue el formato Cormen:

- Use leftarrow para asignaciones (puede escribir <-).
- Los bloques de código se delimitan por **indentación**.
- Las funciones deben tener formato NOMBRE(parámetros).

? ¿Cómo escribo el símbolo leftarrow?

Puede usar cualquiera de estas opciones:

- ← (copiar y pegar)
- <- (el sistema lo convierte automáticamente)

Información de Contacto

Rol	Nombre
Desarrolladores	• Daner Alejandro Salazar Colorado • Jaime Andres Cardona Diaz
Repositorio	EffiCode-Analyzer
Curso	Análisis y Diseño de Algoritmos